

4 The Greatest Common Divisor

One of the most fundamental calculations in number theory is that of the greatest common divisor of two integers. Let u and v be positive integers; it is easy to see (see Exercise 2.1) that if the prime factorization of u is $\prod_{1 \leq i \leq k} p_i^{e_i}$ and the prime factorization of v is $\prod_{1 \leq i \leq k} p_i^{f_i}$, then

$$\gcd(u, v) = \prod_{1 \leq i \leq k} p_i^{\min(e_i, f_i)}. \quad (4.1)$$

Unfortunately, a computation based on equation (4.1) appears to be intractable, since it uses the factorizations of u and v . Therefore, a different approach is necessary.

In this chapter we will examine three algorithms for the greatest common divisor: the Euclidean algorithm, the least-remainder Euclidean algorithm, and the binary algorithm. We discuss a modification known as the extended Euclidean algorithm. Finally, we discuss algorithms to compute gcd-free bases.

4.1 The Euclidean Algorithm

The Euclidean algorithm to compute the greatest common divisor of two integers is based on the following observation:

$$d \mid u \text{ and } d \mid v \quad \text{iff} \quad d \mid v \text{ and } d \mid u \bmod v. \quad (4.2)$$

(Prove this, using the definition in (2.1).) Thus $\gcd(u, v) = \gcd(v, u \bmod v)$.

Now suppose $u_1 > 0$. Then $u_1 > u_0 \bmod u_1$, so if we set $u_2 = u_0 \bmod u_1$, we see from (4.2) that $\gcd(u_0, u_1) = \gcd(u_1, u_2)$; furthermore, $u_1 > u_2$. We now continue this process, setting $u_3 = u_1 \bmod u_2$, and noting that $\gcd(u_1, u_2) = \gcd(u_2, u_3)$. This process cannot continue indefinitely, for $u_1 > u_2 > u_3 > \dots$ and so eventually we must find $u_{k+1} = 0$. Since $\gcd(u_k, 0) = u_k$, we conclude that $\gcd(u_1, u_2) = u_k$.

For example, we have $\gcd(180, 146) = \gcd(146, 34) = \gcd(34, 10) = \gcd(10, 4) = \gcd(4, 2) = \gcd(2, 0) = 2$.

This process can be made into an algorithm as follows:

EUCLID(u, v)

- (1) if ($v = 0$) then
 - (2) return(u)
 - (3) else
 - (4) return(EUCLID($v, u \bmod v$))
-

We have proved

THEOREM 4.1.1 The algorithm EUCLID terminates on integer inputs u, v with $v > 0$, and returns $\gcd(u, v)$.

Although in our analysis we have assumed $v > 0$, by modifying line (2) of the algorithm to read

$$(2) \text{ return } (|u|)$$

we get an algorithm that computes the greatest common divisor for *all* integers u and v .

The naive argument given above shows that the Euclidean algorithm terminates in $O(u)$ steps on input (u, v) , which is not a polynomial time bound (in $\lg u$). However, we will show in the next section that the Euclidean algorithm runs in time polynomial in $\lg u$ and $\lg v$.

4.2 The Euclidean Algorithm: Worst-Case Analysis

We now examine the Euclidean algorithm on input (u, v) , at each step keeping track of the quotients as well as the remainders. Let $u_0 = u$ and $u_1 = v$. Then we have

$$\begin{aligned} u_0 &= a_0 u_1 + u_2 \\ u_1 &= a_1 u_2 + u_3 \\ &\vdots \\ u_{n-2} &= a_{n-2} u_{n-1} + u_n \\ u_{n-1} &= a_{n-1} u_n, \end{aligned} \tag{4.3}$$

where $a_i = \lfloor u_i / u_{i+1} \rfloor$ for all i .

In the previous section, we showed that $u_n = d = \gcd(u, v)$. Notice that the last quotient, a_{n-1} , is always at least 2 (unless $n = 1$); for if $a_{n-1} = 1$ we would have $u_{n-1} = u_n$, and hence $u_{n-2} \bmod u_{n-1} = u_{n-1}$, a contradiction.

Let us call the integer n the number of *division steps* in the Euclidean algorithm; we define $E(u, v) = n$.

We are now ready for the goal of this section: proving a good upper bound on $E(u, v)$.

Let F_n denote the n -th *Fibonacci number*, defined by $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$.

We have

LEMMA 4.2.1 Let u, v be integers such that $u > v > 0$, and the Euclidean algorithm on input (u, v) performs n division steps. Then $u \geq F_{n+2}$ and $v \geq F_{n+1}$.

Proof Let $u = u_0$ and $v = u_1$, and refer to the system of equations (4.3). We will prove by induction on n that $u_0 \geq F_{n+2}$ and $u_1 \geq F_{n+1}$.

This is true for $n = 1$. For then the entire Euclidean algorithm consists of one division, $u_0 = a_0 u_1$, and since $u_0 > u_1$, to find the least u_0, u_1 , we must take $u_1 = 1, a_0 = 2$, and hence $u_0 = 2$.

Now assume the statement is true for all $i < n$; we wish to prove it for n . Then the first step of the Euclidean algorithm sets $u_0 = a_0 u_1 + u_2$, and we know that $E(u_1, u_2) = n - 1$. Thus $u_1 \geq F_{n+1}$ and $u_2 \geq F_n$ by induction. Then $u_0 \geq u_1 + u_2$; hence $u_0 \geq F_{n+2}$. ■

We now wish to determine the worst-case behavior of the Euclidean algorithm. More precisely, for each n , we would like to find the “smallest” input that forces the algorithm to perform n division steps. However, since the algorithm actually takes a *pair* of inputs (u, v) , it is not immediately clear how to define the “smallest” input.

We will say that the pair (u, v) is *lexicographically less* than the pair (u', v') if $u < u'$, or if $u = u'$ and $v < v'$.

COROLLARY 4.2.2 (LAMÉ) Let u, v be integers such that $u > v > 0$, the Euclidean algorithm on input (u, v) performs n division steps, and (u, v) is lexicographically least among all pairs satisfying these properties. Then $(u, v) = (F_{n+2}, F_{n+1})$.

Proof From Lemma 4.2.1, we know that $u \geq F_{n+2}$ and $v \geq F_{n+1}$. To prove the corollary, we must verify that if $u_0 = F_{n+2}$ and $u_1 = F_{n+1}$, then in fact the Euclidean algorithm performs n division steps; actually, it produces the series of quotients $a_0 = 1, a_1 = 1, \dots, a_{n-2} = 1, a_{n-1} = 2$. This amounts to proving that $\lfloor F_{k+2}/F_{k+1} \rfloor = 1$ and $F_{k-1} = F_{k+1} \bmod F_k$ for $k \geq 2$, which is left to the reader as Exercise 4. ■

The reader is asked to prove in Exercise 3 that

$$F_n = \frac{\alpha^n - \beta^n}{\sqrt{5}} \quad (4.4)$$

where $\alpha = (1 + \sqrt{5})/2, \beta = (1 - \sqrt{5})/2$.

From this, we prove

COROLLARY 4.2.3 Let $u > v > 0$. Then $E(u, v) < c_1 \log u + c_2 - 2 + c_3 u^{-1}$, where $c_1 = 1/\log \alpha \doteq 2.08, c_2 = (\log \sqrt{5})/(\log \alpha) \doteq 1.67$, and $c_3 = 1/(\sqrt{5} \log \alpha) \doteq .93$. Also, $E(u, v) < c_1 \log v + c_2 - 1 + c_3 v^{-1}$.

Proof Assume $u \geq F_{n+2}$. Then

$$u \geq \frac{\alpha^{n+2} - \beta^{n+2}}{\sqrt{5}} > \frac{\alpha^{n+2} - 1}{\sqrt{5}}.$$

Hence $(n + 2) \log \alpha \leq \log(1 + u\sqrt{5})$. Now we use the fact that $\log(x + 1) = \log x +$

$\log(1 + 1/x)$ and the estimate $\log(1 + 1/x) < 1/x$ to obtain

$$(n + 2) \log \alpha < \log(u\sqrt{5}) + 1/(u\sqrt{5}),$$

and we see $n < c_1 \log u + c_2 - 2 + c_3 u^{-1}$. Taking the contrapositive, we see that if

$$n \geq c_1 \log u + c_2 - 2 + c_3 u^{-1},$$

then $u < F_{n+2}$. To make the conclusion $u < F_{n+2}$ true, we choose

$$n = \lceil c_1 \log u + c_2 - 2 + c_3 u^{-1} \rceil.$$

From Lemma 4.2.1, if $u < F_{n+2}$, then $E(u, v) < n$. But $E(u, v)$ is an integer, so in fact $E(u, v) < c_1 \log u + c_2 - 2 + c_3 u^{-1}$.

The inequality $E(u, v) < c_1 \log v + c_2 - 1 + c_3 v^{-1}$ follows in a similar fashion. ■

Using Corollary 4.2.3, we easily get a polynomial-time bound on the bit complexity of the Euclidean algorithm on inputs u, v . For all the operations are on integers $\leq u$, and the algorithm performs $O(\lg u)$ divisions on integers with $O(\lg u)$ bits. This gives a time bound of $O((\lg u)^3)$.

With just slightly more work, however, we can improve this bound to $O((\lg u)(\lg v))$. Suppose $u = u_0$ and $v = u_1$ are the inputs, and refer to the system of equations (4.3). Now dividing u_i by u_{i+1} to get a quotient of a_i and a remainder of u_{i+2} can be done in $O((\lg a_i)(\lg u_{i+1}))$ bit operations. Thus the total bit complexity (disregarding constant factors) is

$$\begin{aligned} \sum_{0 \leq i \leq n-1} (\lg a_i)(\lg u_{i+1}) &\leq (\lg v) \sum_{0 \leq i \leq n-1} \lg a_i \\ &\leq (\lg v) \left(n + \log_2 \left(\prod_{0 \leq i \leq n} a_i \right) \right). \end{aligned}$$

Now from Corollary 4.2.3 we know $n = O(\lg u)$, and by Exercise 5, we have $a_0 a_1 \cdots a_{n-1} \leq u$; hence we conclude that the bit complexity is $O((\lg u)(\lg v))$. We have shown

COROLLARY 4.2.4 Let u, v be integers. We can compute the greatest common divisor of u and v using $O((\lg u)(\lg v))$ bit operations.

4.3 The Extended Euclidean Algorithm

The extended Euclidean algorithm takes as input a pair (u, v) and computes $d = \gcd(u, v)$ and a pair of integers (a, b) such that $au + bv = d$.

It is based on the following theorem:

THEOREM 4.3.1 Let u, v , and c be integers. Then the equation

$$au + bv = c$$

has a solution in integers (a, b) iff $\gcd(u, v) \mid c$.

Proof Let d be any integer such that $d \mid u$ and $d \mid v$. Then clearly $d \mid c$; hence $\gcd(u, v) \mid c$.

To prove the other direction, it clearly suffices to show that the equation $au + bv = \gcd(u, v)$ has a solution. This is done in the next theorem.

THEOREM 4.3.2 Let u and v be integers, and let $d = \gcd(u, v)$. Then there exist integers a and b such that

$$au + bv = d.$$

Proof We apply the Euclidean algorithm to $u = u_0$ and $v = u_1$, obtaining the sequence of quotients $a_0, a_1, \dots, a_{n-1}$ as in (4.3). Now we rewrite the system of equations (4.3) as follows:

$$\begin{aligned} \begin{bmatrix} u_0 \\ u_1 \end{bmatrix} &= \begin{bmatrix} a_0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}; \\ \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} &= \begin{bmatrix} a_1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_2 \\ u_3 \end{bmatrix}; \\ &\vdots \\ \begin{bmatrix} u_{n-2} \\ u_{n-1} \end{bmatrix} &= \begin{bmatrix} a_{n-2} & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_{n-1} \\ u_n \end{bmatrix}; \\ \begin{bmatrix} u_{n-1} \\ u_n \end{bmatrix} &= \begin{bmatrix} a_{n-1} & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u_n \\ u_{n+1} \end{bmatrix}. \end{aligned}$$

where $d = u_n = \gcd(u_0, u_1)$ and $u_{n+1} = 0$.

Next, we combine these equations to get

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} a_0 & 1 \\ 1 & 0 \end{bmatrix} \cdots \begin{bmatrix} a_{n-1} & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} d \\ 0 \end{bmatrix}.$$

Now define

$$M_k = \begin{bmatrix} b_k & c_k \\ d_k & e_k \end{bmatrix} = \begin{bmatrix} a_0 & 1 \\ 1 & 0 \end{bmatrix} \cdots \begin{bmatrix} a_k & 1 \\ 1 & 0 \end{bmatrix}.$$

Then we have

$$M_{n-1}^{-1} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} d \\ 0 \end{bmatrix}.$$

Since $\det M_{n-1} = (-1)^n$, we see that

$$M_{n-1}^{-1} = (-1)^n \begin{bmatrix} e_{n-1} & -c_{n-1} \\ -d_{n-1} & b_{n-1} \end{bmatrix}.$$

Hence $e_{n-1}u - c_{n-1}v = (-1)^n d$, and we may take $a = (-1)^n e_{n-1}$, $b = (-1)^{n+1} c_{n-1}$. ■

The proof we have just given for the existence of a and b actually provides an efficient algorithm for calculating them. We have the following algorithm:

EXTENDED EUCLID(u, v)

{ Computes $d = \gcd(u, v)$ and a, b such that $au + bv = d$ }

$$M \leftarrow \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$n \leftarrow 0$

while ($v \neq 0$) do

$$q \leftarrow \lfloor u/v \rfloor \quad \{ \text{compute } a_n \}$$

$$M \leftarrow M \begin{bmatrix} q & 1 \\ 1 & 0 \end{bmatrix}$$

$$(u, v) \leftarrow (v, u - qv)$$

$$n \leftarrow n + 1$$

return($d = u, a = (-1)^n M_{22}, b = (-1)^{n+1} M_{12}$)

COROLLARY 4.3.3 Given integers u and v , we can find integers a, b such that

$$au + bv = d$$

in $O((\lg u)(\lg v))$ bit operations.

Proof The algorithm computes the matrices $M_0, M_1, \dots, M_{n-1}$. It is easy to see that

$$b_0 \leq b_1 \leq b_2 \cdots \leq b_{n-1} \leq u,$$

and similarly for the other entries of the M_i . Let us compute the number of bit operations required to compute the b_i , the other entries of the matrices M_i being handled similarly. Since $b_{i+1} = b_i a_{i+1} + c_i$ for $0 \leq i \leq n-2$, the total number of bit operations to compute

all the b_i (disregarding constant factors) is bounded by

$$\sum_{0 \leq i \leq n-2} (\lg b_i)(\lg a_{i+1}) + (\lg b_i) + (\lg a_{i+1}) + (\lg c_i).$$

Using exercise 5 and Corollary 4.2.3, it is easy to see that this sum is $O((\lg u)(\lg v))$.

An example: let $u = 180$, $v = 146$. Then the quotients in the Euclidean algorithm are $a_0 = 1$, $a_1 = 4$, $a_2 = 3$, $a_3 = 2$, and $a_4 = 2$. We find $n = 5$ and

$$M_4 = \begin{bmatrix} 90 & 37 \\ 73 & 30 \end{bmatrix}.$$

Hence $a = -30$, $b = 37$, and $au + bv = 2$.

4.4 The Euclidean Algorithm and Continuants

Notice that, in the system of equations (4.3), each u_i is actually a multiple of $d = \gcd(u_0, u_1) = u_n$. For example,

$$u_{n-1} = a_{n-1}u_n = a_{n-1}d,$$

$$u_{n-2} = a_{n-2}u_{n-1} + u_n = (a_{n-2}a_{n-1} + 1)d$$

and

$$u_{n-3} = a_{n-3}u_{n-2} + u_{n-1} = (a_{n-3}a_{n-2}a_{n-1} + a_{n-3} + a_{n-1})d;$$

this suggests writing each u_i as the product of d and a polynomial in the a_i .

These polynomials are classical objects called *continuants*. Continuants are defined as follows:

DEFINITION 4.4.1

$$Q_0() = 1$$

$$Q_1(X_0) = X_0$$

(4.5)

$$Q_{n+1}(X_0, X_1, \dots, X_n) = X_n Q_n(X_0, X_1, \dots, X_{n-1}) + Q_{n-1}(X_0, X_1, \dots, X_{n-2})$$

for $n \geq 1$.

We will see below in Theorem 4.4.5 that

$$u_i = dQ_{n-i}(a_i, a_{i+1}, \dots, a_{n-1}).$$

4.10 Notes on Chapter 4

It is not known if the gcd of n -bit inputs can be computed in parallel with a polynomial number of processors and depth significantly less than $O(n)$; this question was first raised by Cook [1985]. Kannan, Miller, and Rudolph [1987] showed how to compute the gcd on n bit inputs in $O(n(\lg \lg n)/\lg n)$ parallel time using $O(n^2(\lg n)^2)$ processors. Chor and Goldreich [1990] improved this to $O(n/\lg n)$ time and $O(n^{1+\epsilon})$ processors. Also see Sorenson [1994]. Adleman and Kompella [1988] showed how to compute the extended gcd in polylogarithmic depth and subexponential size.

Mansour, Schieber, and Tiwari [1991a] gave a $\Omega(\log \log n)$ lower bound (in the algebraic decision tree model) for any algorithm to compute the gcd using the four field operations, mod, and comparisons. A related result is in Mansour, Schieber, and Tiwari [1991b]. Also see Deng [1989]; Lin-Kriz and Pan [1992].

A. Zavrotsky invented an analogue computer for finding the greatest common divisor based on the result in Exercise 17; see Gardner [1971]; Zavrotsky [1961].

4.1

In the literature, the greatest common divisor of u and v is frequently written (u, v) , a notation we have avoided in this book.

The Euclidean algorithm can be found in Euclid's *Elements*, Book VII, Proposition 2; for example, see Heath [1956]. For an extensive discussion of the Euclidean algorithm, see Bachmann [1902, pp. 99–153]. For algorithms to compute the gcd of more than two inputs, see Rosser [1942]; Ficken [1943]; Weinstock [1960]; Blankinship [1963]; Bradley [1970]; Waterman [1977]; and Majewski and Havas [1994]; Havas, Majewski, and Matthews [1995]. Barlow [1811, p. 22] compared computing the greatest common divisor of two or more numbers via the prime factorization of the given numbers; he wrote

... this method [via factorization] is, however, rather theoretical than practical, being by no means so ready in application as the rule generally given for this purpose in books of arithmetic.

In practice, the Euclidean algorithm may be sped up considerably by using the technique of *loop unrolling*; for more on this, see Bentley [1982].

P. Cohn [1961] proposed a very general form of the Euclidean algorithm.

4.2

Theorem 4.2.2 is traditionally credited to Lamé [1844]. However, seven years earlier it appeared without proof in a paper of Léger [1837]. Also, de Lagny [1733, pp. 363–364] discussed the “simplest” rational numbers having a continued fraction of a given length.

Finck [1841, pp. 44–45] observed that if $u > v > 0$, then $u \bmod v < u/2$. From this it follows, he remarked, that the number of steps $E(u, v)$ performed by the Euclidean algorithm is bounded above

by $2 \log_2 v + 1$. This analysis is reproduced in Finck [1842] and Finck [1843, pp. 57–58]. Other remarks about Lamé's theorem can be found in Finck [1845]; von Grosschmid [1911]; Grossman [1924]; Thoro [1964a, 1964b]; V. Harris [1970b]; Meijer [1992]. See Shallit [1994] for the history of early analyses of the Euclidean algorithm.

Lamé's theorem was originally stated as follows: the number of division steps in the Euclidean algorithm on input (u, v) is bounded above by 5 times the number of decimal digits in $\min(u, v)$.

Theorem 4.2.4 was first proved by G. Collins [1968]. G. Collins [1974a] also obtained similar results on the average bit complexity of the Euclidean algorithm.

D. H. Lehmer [1938] proposed an interesting method of speeding up the Euclidean algorithm by doing most of the divisions with single precision numbers. Also see Jebelean [1993c]. However, the binary gcd algorithm appears to be significantly faster in practice. Sorenson [1995] analyzed a variant of Lehmer's method, and proved it uses $O(n^2 / \log n)$ bit operations on inputs of n bits.

The average-case complexity of the Euclidean algorithm (counting the number of division steps) was first discussed by Heilbronn [1969]. Further work was done by Dixon [1970, 1971]; Tonkov [1974]; Kilian [1983]; and G. Norton [1990]. Letting

$$t(n) = \frac{1}{\varphi(n)} \sum_{\substack{1 \leq k \leq n \\ \gcd(k, n) = 1}} E(n, k),$$

Porter [1975] showed that

$$t(n) = \frac{12 \log 2}{\pi^2} \log n + C + O(n^{-1/6+\epsilon}).$$

Knuth [1976b] proved the following expression for C :

$$C = \frac{6 \log 2}{\pi^2} (3 \log 2 + 4\gamma - 24\pi^2 \zeta'(2) - 2) - \frac{1}{2}.$$

More recently, Hensley [1992] has shown that the number of division steps done by the Euclidean algorithm over all pairs (u, v) with $0 < u \leq v \leq x$ is asymptotically normally distributed, with mean close to $12(\log 2)\pi^{-2} \log x$.

Plankensteiner [1970] counted the number of pairs (u, v) for which $E(u, v) = k$.

A subtractive version of the Euclidean algorithm was analyzed by Yao and Knuth [1975]; Pitteway and Castle [1988]; Zheng [1994]. Also see Knopfmacher [1992]. Brun [1964] discussed the relationship between the subtractive algorithm and musical theory.

Using the Schönhage-Strassen fast multiplication method, Knuth [1971] gave an algorithm for the gcd that uses $O(n(\lg n)^5 \lg \lg n)$ steps on n bit integer inputs. This was improved by Schönhage [1971] to $O(n(\lg n)^2 \lg \lg n)$. A generalization was given by Moenck [1973]. See Schönhage, Grotfeld, and Vetter [1994] for a discussion of a practical implementation of this method.

An integral domain D in which there exists a map $\delta : D \rightarrow \mathbb{N}$ such that if a, b are nonzero elements of D , then there exist q, r in D such that $a = qb + r$ with $\delta(r) < \delta(b)$, is called *Euclidean*. A well-known theorem says that all Euclidean domains have unique factorization. For example, $\mathbb{Z}$ and

$\mathbb{Z}[i]$ (the Gaussian integers) are Euclidean domains. Samuel [1971] and P. Cohn [1991] discussed the properties of Euclidean domains. For the Euclidean algorithm in real quadratic fields, see Chatland [1949]; Chatland and Davenport [1950]. Also see Kaltofen and Rolletschek [1989] and Rolletschek [1990].

Hurwitz [1887] gave a Euclidean algorithm for $\mathbb{Z}[i]$; also see Caviness and Collins [1976]; Shallit [1979b]. The analogue of the Kronecker-Vahlen theorem for $\mathbb{Z}[i]$ was given by Rolletschek [1986].

Other papers on computational aspects of the gcd include Nievengloski [1849]; Daykin [1970]; Shea [1973]; V. Harris [1974]; Schaefer [1975]; van Trigt [1978]; Abramov [1979]; Marcus [1981]; Epasinghe [1985]; Bojanczyk and Brent [1987]; C. Purdy and Purdy [1990a, 1990b]; Yang [1990]. Mays [1987] discusses some variations on the Euclidean algorithm.

4.3

The first explicit solution to $au + bv = d$ appears in the Sanskrit astronomical work Aryabhatiya, of the 5th–6th century A.D.; see Datta and Singh [1938] or Weil [1984].

Blum and Kannan [1989, 1995] showed how to design a “self-checking” program for the extended gcd.

4.4

Continuants were introduced by Euler [1762]. Our presentation, based on 2×2 matrices seems to have originated with Hurwitz; see Hurwitz and Kritikos [1986]. It was rediscovered by Frame [1949] and Kolden [1949], independently. The term “continuant” was invented by T. Muir [1874]. Also see Muir [1878].

4.5

The material in this section is based on G. Hardy and Wright [1985]. For the history of continued fractions, see Brezinski [1991].

Many authors have been concerned with generalizing the continued fraction algorithm to dimensions $n > 2$. We list just a few of the relevant references here: L. Bernstein [1971]; Ferguson and Forcade [1979, 1982]; Brentjes [1981]; Ferguson [1986, 1987]; Hastad, Just, Lagarias and Schnorr [1989]; and Lagarias [1994].

4.6

The least-remainder algorithm was first analyzed by Binet [1841, p. 454]. Later remarks were given by Nievengloski [1845]. Theorem 4.6.2 is due to Dupré [1846]. Also see Brown and Duncan [1971].

Theorem 4.6.4 was stated by Kronecker [1901, p. 118]; it was first proved by Vahlen [1895]. Our presentation is based on Uspensky and Heaslet [1939]. Lazard [1977] generalized the situation of Theorem 4.6.4 to cover the case where *any* remainder is chosen at each step of the algorithm.

A relationship between the ordinary Euclidean algorithm and the least-remainder algorithm was discussed by Goodman and Zaring [1952]. Also see T. Moore [1992].

4.7

The binary gcd algorithm was invented by J. Stein [1967]. Using a heuristic model, it was analyzed by Knuth [1969, 1981] and Brent [1976a]. Similar algorithms were discussed by Brent, Kung, and Luk [1983]; Brent and Kung [1983, 1985]; G. Purdy [1983]; Norton [1985]; Yun and Zhang [1986]; Norton [1987]; Mandelbaum [1988]; Parikh and Matula [1991]; Shallit and Sorenson [1994]. Sorenson [1994] developed generalizations of the binary algorithm, called the k -ary gcd algorithm, and proved a running time of $O(n^2/\log n)$ on n -bit inputs. A similar algorithm was studied by Jebelean [1993b]; also see Weber [1995]. A cross between the Euclidean algorithm and the binary gcd algorithm was proposed by V. Harris [1970a]. In Alagić and Arbib [1978, pp. 223–226], a polynomial-time binary gcd algorithm is transformed through the magic of top-down program design into an exponential-time algorithm.

The simplicity of the operations used in the binary gcd algorithm make it the method of choice on real computers. For example, J. Sorenson (personal communication) found that the binary method was between 10 and 25 times as fast as the Euclidean algorithm on a DECstation 3100, on inputs of 10 to 1000 digits in size. Also see the article of Norton [1985], which found a similar, though less dramatic improvement on large inputs. However, different results were obtained by Jebelean [1993a].

4.8

Stieltjes [1890, §19] appears to have been the first to study an algorithm similar to our gcd-free basis algorithm. He was interested in expressing $\text{lcm}(a, b) = AB$, where A and B are integers satisfying the conditions $A \perp B$, $A \mid a$, and $B \mid b$. (A and B can also be computed efficiently with a gcd-free basis.) Later, Dedekind [1897] studied factorization by repeated use of the Euclidean algorithm. However, his construction took exponential time.

Several authors have considered constructing a gcd-free basis for polynomials; see Collins [1974b]; P. S. Wang [1980]. Also see von zur Gathen [1984b, 1986b] and Kaltofen [1985a, 1985b].

For such algorithms on integers, see Bach, Miller, and Shallit [1984, 1986]. Theorems 4.8.6 and 4.8.7 were first proved by Bach, Driscoll, and Shallit [1990, 1993]. The proof we have presented is due to H. W. Lenstra, Jr. Our presentation of some of the material in this section, particularly Theorem 4.8.5, is based on a manuscript of D. Bernstein [1992].

Other applications of the gcd-free basis construction can be found in H. W. Lenstra [1992]; Ge [1993]; Buchmann and Lenstra [1994].